

# DocDock Local API Documentation

## Developer Reference Guide

DocDock local API allows developers to programmatically submit local document files or links for remote documents to be processed by user's chosen AI models (Gemini or GPT). It parses document structures against a picked schema defined by user on DocDock platform and returns cleanly structured JSON results.

### Dynamic Port Allocation

DocDock as a local application, has its port dynamically assigned on startup. Users can locate the active port directly from the "account dashboard" on DocDock's desktop app. Access the API via your current active port.

### API Endpoint Reference

**POST** `http://localhost:<PORT>/api/convert`

| Parameter        | Type   | Required | Description                                                                          |
|------------------|--------|----------|--------------------------------------------------------------------------------------|
| <b>workspace</b> | String | Required | Target workspace for your document as in DocDock desktop app.                        |
| <b>template</b>  | String | Required | Target template for your document as in DocDock desktop app.                         |
| <b>file_path</b> | String | Required | Absolute local file path (e.g. C:\path\doc.pdf) OR a public remote URL (HTTP/HTTPS). |

| HTTP Status | Error Code Text          | Root Cause / Resolution                                                                         |
|-------------|--------------------------|-------------------------------------------------------------------------------------------------|
| 400         | NO_FILE_PROVIDED         | file_path missing                                                                               |
| 400         | FILE_READ_FAILED         | Failed to read file. Check file path formatting and file access permissions.                    |
| 404         | TEMPLATE_NOT_FOUND       | The requested combination of workspace and template does not exist.                             |
| 403         | NO_ACTIVE_API_CREDENTIAL | No LLM provider keys (Gemini / OpenAI) have been selected or activated inside DocDock settings. |
| 422         | MODEL_OUTPUT_INVALID     | The AI engine executed successfully, but its response failed DocDock schema validation.         |

**POST** `http://localhost:<PORT>/api/publish`

| Parameter              | Type   | Required | Description                                                                                                                                                                          |
|------------------------|--------|----------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>connection_name</b> | String | Required | The target connection name for AWS S3, Azure Blob or Google GCS.                                                                                                                     |
| <b>zip_type</b>        | String | Required | Compression data type format. Must be either 'JSON' or 'CSV'.                                                                                                                        |
| <b>file_results</b>    | Array  | Required | List of conversion results for your files to zip and publish:<br><pre>[{<br/>  path: "your/original/file_path.txt",<br/>  result: &lt;json result from /api/convert&gt;<br/>}]</pre> |

| HTTP Status | Error Code Text           | Root Cause / Resolution                                               |
|-------------|---------------------------|-----------------------------------------------------------------------|
| <b>400</b>  | NO_FILE_PROVIDED          | file_results list is empty or missing.                                |
| <b>400</b>  | INVALID_ZIP_TYPE          | The provided zip_type is not supported (must be 'JSON' or 'CSV').     |
| <b>400</b>  | NO_VALID_DATA             | Failed to zip results in file_results                                 |
| <b>400</b>  | INVALID_CONNECTION_CONFIG | The provided connection_name has invalid configuration                |
| <b>404</b>  | CONNECTION_NOT_FOUND      | The provided connection_name does not match any existing connections. |
| <b>500</b>  | PUBLISH_ERROR             | Internal server error during processing or cloud platform uploading.  |

## Python Integration Example

The Python script below demonstrates conversion for a list of file paths (local & online) using [/api/convert](#) and publishing their results to an AWS S3 connection using [/api/publish](#).

```
import urllib.request
import urllib.parse
import urllib.error
import json

#####
#                                     #
#      YOUR CONFIGURATION           #
#                                     #
#####

PORT = 3000
WORKSPACE = "YOUR_WORKSPACE"
TEMPLATE = "YOUR_TEMPLATE"
CONNECTION_NAME = "sampleAwsS3"
ZIP_TYPE = "CSV" # or "ZIP"

#####
#                                     #
#      YOUR FILE TO PROCESS         #
#                                     #
#####

file_paths = [
    "C:\Users\JohnDoe\Desktop\test1.pdf",
    "C:\Users\JohnDoe\Desktop\test2.pdf",
    "https://s3-eu-west-2.amazonaws.com/example-bucket/live_sample.jpg"
]
processed_files = []

#####
#                                     #
#      FILES PROCESSING             #
#      AND                          #
#      RESULT RECORDING             #
#                                     #
#####

for file_path in file_paths:
    print(f"PROCESSING FOR {file_path}")
    try:
        query = urllib.parse.urlencode({
            "workspace": WORKSPACE,
            "template": TEMPLATE,
            "file_path": file_path,
```

```

    })
    url = f"http://localhost:{PORT}/api/convert?{query}"
    req = urllib.request.Request(url, method="POST")
    with urllib.request.urlopen(req) as response:
        body = response.read().decode("utf-8")
        print(f"==> FILE COMPLETED\n")
        processed_files.append({"path": file_path, "result": json.loads(body)})
    except urllib.error.URLError as e:
        print(f"==> FILE ERROR: {e}\n")

#####
#                                     #
#      PUBLISH RESULTS TO           #
#      YOUR CONNECTION              #
#                                     #
#####

print(f"PUBLISHING {len(processed_files)} FILE(S)")
try:
    publish_body = json.dumps({
        "connection_name": CONNECTION_NAME,
        "zip_type": ZIP_TYPE,
        "file_results": processed_files,
    }).encode("utf-8")
    publish_req = urllib.request.Request(
        f"http://localhost:{PORT}/api/publish",
        data=publish_body,
        method="POST",
        headers={"Content-Type": "application/json"},
    )
    with urllib.request.urlopen(publish_req) as response:
        publish_result = response.read().decode("utf-8")
        print(f"==> PUBLISHED TO: {publish_result}")
    except urllib.error.URLError as e:
        print(f"==> PUBLISH ERROR: {e}")

```